# Day 5 – Gesture Recording Setup & Camera Troubleshooting

**Date:** 05/02/2026
**Duration:** 3 hr 50 min (21:15 – 01:05)

---

## 1. Objective of Day 5

- Start collecting Ethiopian Sign numbers (0–20) with the camera.
- Verify that the existing `collect_landmarks.py` works for recording landmarks to CSV.
- Test live hand landmark detection with `hand_landmark_live.py`.
- Integrate DroidCam as an alternative camera due to low-quality laptop webcam.
- Ensure the pipeline works for both recording and visualization.

---

## 2. What We Did / Attempted

1. Opened `collect_landmarks.py` and verified that the script correctly saves CSVs when recording.
2. Opened `hand_landmark_live.py` to see green landmark dots on hands; confirmed visualization works with original webcam.
3. Installed **DroidCam** and connected it to the laptop via WIFI.
4. Attempted to use DroidCam as input to OpenCV using:

```
cap = cv2.VideoCapture(1)
```

5. Encountered **green screen / no image** issues.
6. Tried different OpenCV backends: `CAP_DSHOW` and `CAP_MSMF`.
7. Attempted HTTP stream method with DroidCam's IP URL, but OpenCV gave errors (`CV_IMAGES` backend failure).
8. Realized the errors were due to device locking — the DroidCam client was occupying the camera driver.
9. Tested camera independently with a minimal OpenCV script to find the working index.

---

## 3. What Went Wrong / Specific Errors

- **Green screen**: caused by incorrect OpenCV backend selection.

- **HTTP stream errors**: OpenCV treating the URL as an image sequence due to backend fallback.
- **Device busy / unavailable**: OpenCV could not access DroidCam because the Windows client already held the device.

**Error messages observed:**

```
VIDEOIO(DSHOW): backend is generally available but can't be used to capture
by index
VIDEOIO(CV_IMAGES): can't find starting number (in the name of file)
DroidCam is busy or unavailable
```

**Why it went wrong:**

- Windows allows only one client to access a webcam device at a time.
- OpenCV default backend may not work properly with DroidCam over USB/WiFi.
- HTTP stream from DroidCam free version may not be available or blocked by firewall.

---

# 4. Lessons Learned

1. Only one program can access a camera device at a time — always close preview windows and other apps.
2. OpenCV backend selection is critical for external virtual cameras like DroidCam.
3. USB connection is more stable than WiFi for DroidCam when collecting data.
4. Testing the camera independently with a minimal script saves time before integrating with MediaPipe.
5. Python / MediaPipe code works correctly; the issues were **hardware/driver related**, not ML-related.

---

# 5. Next Steps / Plan for Day 6

- Ensure a stable camera connection (USB preferred).
- Identify the correct camera index for DroidCam on the laptop.
- Use `collect_landmarks.py` to start collecting 0–20 gestures with working camera.
- Confirm green dots appear during recording to verify hand detection.
- Continue building the dataset for ML training.
- Document camera setup and environment dependencies for reproducibility.

---

**Day 5 Outcome:** Partial success (camera issue prevented proper recording)
**Day 5 Value:** Learned device handling for virtual cameras, confirmed that landmark recording code works with functioning camera, prepared plan for dataset collection.